

20-frontend-frameworks

Modul 20: Framework Frontend Modern — Konsep Inti, Bukan Tutorial Framework

Target: SMK RPL — siswa sudah paham HTML, CSS, JavaScript dasar

Tujuan: Paham pola arsitektur yang SAMA di semua framework frontend modern

Amanat: Setelah modul ini, siswa bisa belajar React, Vue, Svelte, Solid, atau Qwik manapun — karena konsepnya sama

Daftar Isi

1. Kenapa Ada Framework Frontend?
 2. Component Model — Props In, Events Out
 3. Reactivity — Gimana Framework Tahu Ada Perubahan?
 4. Component Lifecycle — Lahir, Hidup, Mati
 5. State Management — Local, Lifted, Global
 6. Routing — SPA vs SSR
 7. SSR / SSG / ISR — Kapan Pakai Apa?
 8. Tabel Perbandingan Framework
 9. Kesimpulan — Pilih yang Mana?
-

1. Kenapa Ada Framework Frontend?

Dulu bikin web pake HTML + CSS + JavaScript vanilla. Ngambil data dari server? `fetch()`. Update DOM? `document.getElementById('x').innerHTML = ...`. Mantemin state (apakah tombol udah diklik)? bikin variable global. Proyek kecil OK, proyek gede jadi **spaghetti code** — DOM diubah dari 20 tempat beda, state tabrakan, bug susah dilacak.

Framework frontend lahir buat jawab tiga masalah inti:

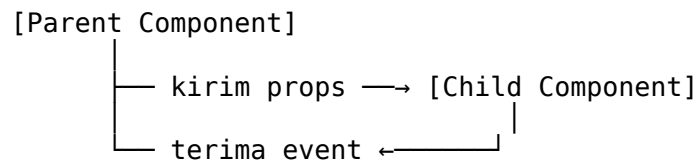
1. **Reactivity** — data berubah → UI otomatis berubah (gausah innerHTML manual)
2. **Component system** — UI dipecah jadi komponen kecil yang reusable
3. **Declarative rendering** — ngomong "ini yang harus tampil", bukan "bagaimana cara ngubah DOM"

Framework modern (React, Vue, Svelte, Solid, Qwik, Angular) beda-beda sintaks dan filosofi, tapi tiga masalah di atas mereka selesaikan **dengan pola yang hampir sama**.

2. Component Model — Props In, Events Out

Setiap framework pake **component** sebagai unit UI terkecil. Component itu fungsi/kelas yang nerima input (disebut **props**), dan ngirim output lewat **events**.

Pola universal (di semua framework):



Props (input, satu arah)

Props itu **read-only dari sisi child**. Child ga boleh ngubah props langsung. Yang bisa ngubah cuma parent.

Framework	Cara definisi props
React	<code>function Card({ title, image })</code>
Vue	<code>defineProps(['title', 'image'])</code>
Svelte	<code>export let title, image (di dalam <script>)</code>
Solid	<code>function Card(props) { props.title }</code>
Qwik	<code>export const Card = component\$((props: {title: string}) => ...)</code>

Events (output, ke atas)

Child ngasih tau parent ada sesuatu terjadi (tombol diklik, form di-submit). Framework handle ini dengan event system masing-masing.

Framework	Cara emit event
React	Child terima fungsi sebagai prop: <code>onClick={props.onSave}</code>
Vue	<code>emit('save', data)</code>
Svelte	<code>createEventDispatcher()</code> atau <code>on:click</code> di parent
Solid	Sama kayak React — props callback
Qwik	<code>\$(props.onSave)</code>

Kenapa pola ini penting?

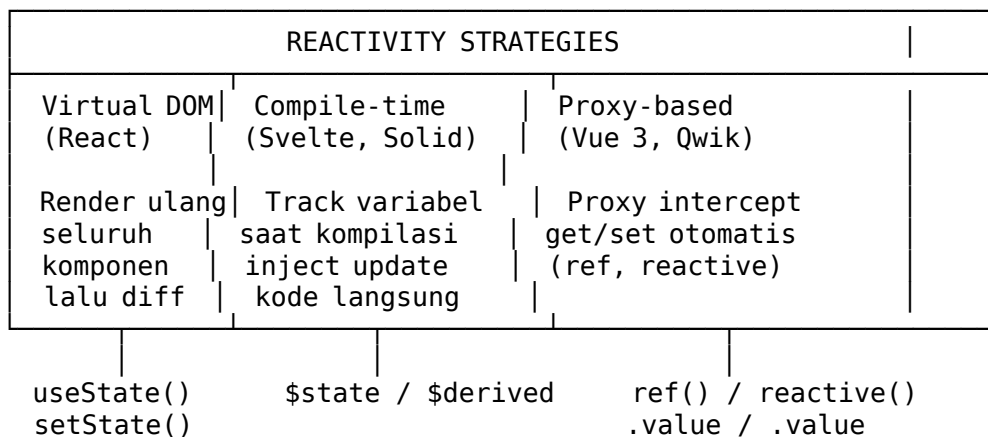
Data cuma mengalir **satu arah**: parent → child lewat props, child → parent lewat events. Ga ada yang ngubah data dari dua arah sekaligus. Ini yang bikin debugging gampang — tinggal liat data datang dari mana.

3. Reactivity — Gimana Framework Tahu Ada Perubahan?

Reactivity adalah **kemampuan framework buat otomatis update UI saat data berubah**. Tanpa reactivity, kita musti manual inner-HTML atau `textContent` tiap kali data berubah.

Setiap framework punya mekanisme beda buat "nge-track" perubahan:

Diagram konseptual



Cara kerja masing-masing

A. Virtual DOM (React, Vue 2)

Framework pake representasi DOM di memory (Virtual DOM). Saat state berubah:

1. Render ulang component → hasilnya Virtual DOM baru
2. **Diff** Virtual DOM baru vs lama
3. Hitung operasi DOM minimal → apply ke real DOM

React pake `useState()` — saat `setState()` dipanggil, React render ulang **seluruh komponen** (plus child-nya kecuali di-memoize).

```
// React
const [count, setCount] = useState(0);
setCount(count + 1); // trigger re-render seluruh komponen
```

B. Compile-time reactivity (Svelte, Solid)

Svelte dan Solid **nganalisis kode saat kompilasi**. Mereka liat variabel mana yang dipake di template, lalu inject kode update yang spesifik ke variabel itu — gausah render ulang seluruh komponen.

```
<!-- Svelte -->
<script>
  let count = $state(0); // $state = reactive signal
  count++;               // Svelte inject update count DI SINI
</script>
<p>{count}</p> <!-- cuma <p> ini yang di-update, bukan seluruh komponen -->
```

Solid mirip Svelte tapi pake `createSignal()`.

C. Proxy-based (Vue 3, Qwik)

Vue 3 pake Proxy JavaScript. Saat properti object diakses (get) atau diubah (set), Proxy otomatis nge-track dependencies dan trigger update.

```
// Vue 3
const count = ref(0);
count.value++; // Proxy detect perubahan → update komponen yang pake count
```

Qwik pake pendekatan mirip — signal + proxy — tapi fokus ke **re-sumability** (ga perlu re-run semua kode saat halaman diload).

Bedanya kunci

Pendekatan	Contoh	Kelebihan	Kekurangan
Virtual DOM	React	Ekosistem gede, prediktif	Overhead render ulang

Pendekatan	Contoh	Kelebihan	Kekurangan
Compile-time	Svelte, Solid	Bundle kecil, performa tinggi	Harus dikompilasi dulu
Proxy-based	Vue 3, Qwik	Reactive otomatis, gampang dipake	Proxy punya edge cases

Lesson: Bedanya cuma **bagaimana** framework detect perubahan. Tujuan akhir SAMA: data berubah → UI berubah. Siswa ga harus hafal detail tiap framework — cukup paham pola dasarnya.

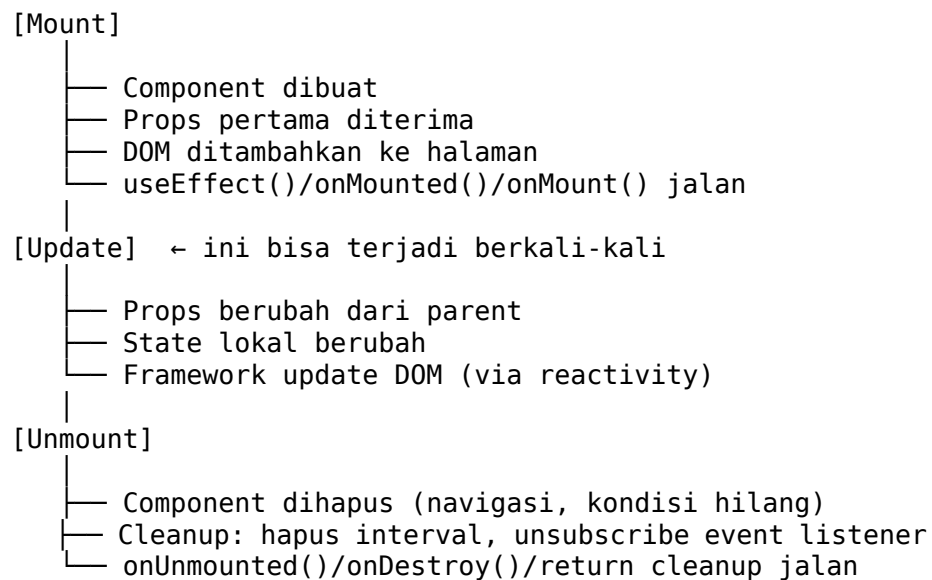
4. Component Lifecycle — Lahir, Hidup, Mati

Setiap component punya siklus hidup: dibuat (mount), di-update, dihapus (unmount). Tiap framework pake istilah dan API beda, tapi konsepnya identik.

Tabel lifecycle mapping

Fase	React	Vue	Svelte
Mount (lahir)	useEffect(fn, [])	onMounted()	onMount()
Update (data berubah)	useEffect(fn, [dep])	watchEffect() / watch()	\$effect()
Unmount (dihapus)	return function di useEffect	onUnmounted()	onDestroy()

Alur visual



Contoh cleanup

Kalo component bikin interval timer, pas component di-unmount interval harus dibersihkan — kalo nggak, memory leak.

```
// React
useEffect(() => {
  const id = setInterval(() => tick(), 1000);
  return () => clearInterval(id); // unmount → cleanup
}, []);

<!-- Vue -->
<script setup>
import { onMounted, onUnmounted } from 'vue'
let id;
onMounted(() => { id = setInterval(tick, 1000) })
onUnmounted(() => clearInterval(id))
</script>

<!-- Svelte -->
<script>
import { onMount, onDestroy } from 'svelte'
let id;
onMount(() => { id = setInterval(tick, 1000) })
onDestroy(() => clearInterval(id))
</script>
```

Konsep kunci: lifecycle hooks itu buat **side effects** ambil data API, event listener, timer, WebSocket. Jangan disalahgunain buat logic biasa — itu tugas reactivity.

5. State Management — Local, Lifted, Global

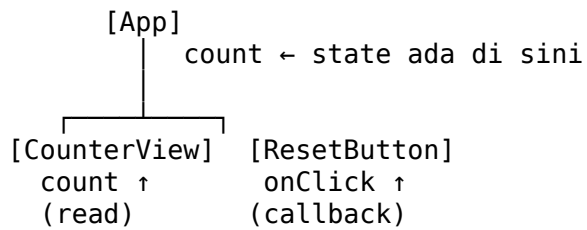
A. Local State

State yang cuma dipake di satu komponen. Contoh: nilai input form, toggle dropdown, counter.

```
<!-- Svelte – local state -->
<script>
  let isOpen = $state(false);
</script>
```

B. Lifted State (Props Drilling)

Dua atau lebih komponen perlu data yang sama. State dinaikin ke parent terdekat, lalu di-pass lewat props.



Ini yang disebut **lifting state up**. Pattern ini ada di semua framework.

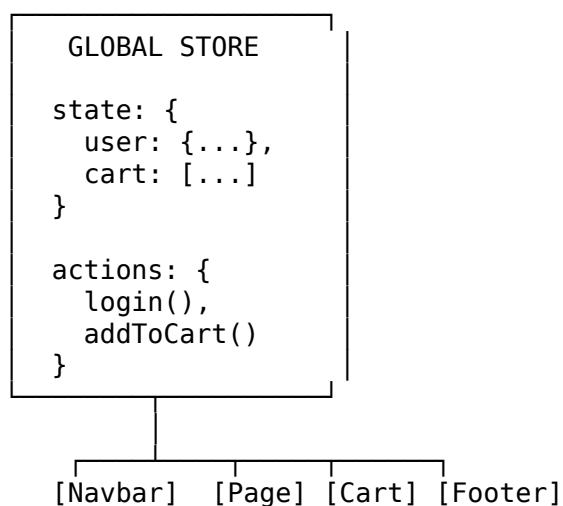
C. Global State (Stores)

Kalo state dipake di banyak tempat yang ga punya parent sama (jaraknya jauh di pohon komponen), lifting ga praktis — musti nge-pass lewat 7 layer komponen (props drilling).

Solusi: **global store** — wadah state yang bisa diakses komponen manapun.

Framework	Global store library bawaan / populer
React	Zustand, Redux, Jotai, Context API
Vue	Pinia (resmi), Vuex (legacy)
Svelte	Svelte stores (bawaan), \$state runes
Solid	createContext + createSignal
Qwik	useContext + useSignal

Konsep store (semua framework sama)



Store punya:

- **State** — data yang di-share
- **Actions** — fungsi yang ngubah state
- **Subscribers** — komponen yang pake state itu; saat state berubah, subscriber otomatis update

Contoh — cara panggil store di berbagai framework:

```
// React + Zustand
const useStore = create((set) => ({
  count: 0,
  increment: () => set((s) => ({ count: s.count + 1 })))
});
function Counter() {
  const count = useStore((s) => s.count);
  // ...
}

// Vue + Pinia
export const useCounterStore = defineStore('counter', () => {
  const count = ref(0);
  function increment() { count.value++ }
  return { count, increment }
});
// di komponen: const store = useCounterStore()

<!-- Svelte store (bawaan) -->
<script>
import { count, increment } from './stores/counter';
</script>
<!-- Di template: {$count} -->
```

Aturan main: Mulai dari local state. Kalo mulai susah di-pass, **lifting state up**. Kalo lifting udah terlalu banyak props drilling, **baru pake global store**. Jangan langsung global store dari awal — itu over-engineering.

6. Routing — SPA vs SSR

SPA (Single Page Application)

Di SPA, **semua kode termasuk halaman” di-load sekali** pas pertama buka. Setelah itu navigasi antar halaman ga reload halaman — cuma konten yang diganti via JavaScript. Ini yang bikin navigasi terasa **cepat** (no flash putih).

Framework punya **client-side router**:

Framework	Router bawaan/populer
React	React Router
Vue	Vue Router (resmi)
Svelte	SvelteKit (file-based)
Solid	Solid Router
Angular	Router bawaan

Cara kerja: URL berubah → router logic jalan → component beda ditampilkan → browser ga reload.

[URL: /produk/123]

Client-side router detect path "/produk/123"

Router panggil komponen `<ProductDetail id={123} />`

Render di dalam `div#root` – gausah request HTML dari server

Kelemahan SPA: SEO jelek (kecuali pake SSR), initial load berat karena harus download JS dulu.

SSR (Server-Side Rendering)

HTML di-render di server, dikirim ke browser, JavaScript tinggal "nyiram" interaktivitas (hydrate). SEO bagus, initial load lebih cepet kelihatan.

Setiap framework punya meta-framework buat SSR:

Client framework	SSR meta-framework
React	Next.js, Remix
Vue	Nuxt
Svelte	SvelteKit
Solid	SolidStart
Qwik	Qwik City

Kapan pilih apa?

- **SPA:** Aplikasi internal (dashboard admin, tools) — butuh interaksi kompleks, SEO ga penting
- **SSR:** Landing page, e-commerce, blog — SEO penting, first load harus cepet

Catatan: Kebanyakan aplikasi modern hybrid — pake SSR buat halaman pertama, lalu SPA buat navigasi selanjutnya. Itu pattern **MPA + SPA** yang sekarang standar di Next.js, Nuxt, SvelteKit.

7. SSR / SSG / ISR — Kapan Pakai Apa?

Di atas SSR, ada metode rendering lain:

SSG (Static Site Generation)

Halaman di-render **pas build time**. Hasilnya HTML statis. Cocok buat konten yang jarang berubah.

```
[Build time]
|
Ambil data dari API / CMS
|
Generate HTML untuk tiap halaman
|
Deploy HTML ke CDN / server
|
User minta halaman → langsung dapet HTML (super cepet!)
```

Contoh: blog, dokumentasi, marketing page.

ISR (Incremental Static Regeneration)

Hybrid SSG + SSR. Halaman di-render statis, tapi bisa di-regenerate di background setelah periode tertentu.

```
[User request /produk/123]
|
CDN serve HTML statis (lama: dari build)
|
┌──────────Setelah 60 detik──────────┐
│ Background: minta API               │
│ Generate ulang HTML                 │
│ Simpan di CDN                       │
└────────────────────────────────────────┘
```

User selanjutnya dapet HTML baru

Tabel keputusan

Skenario	Metode	Kenapa
Dashboard admin	SPA	Cepet navigasi, SEO ga penting
Blog / dokumentasi	SSG	Konten statis, super cepet
Toko: halaman produk	SSG + ISR	Produk ribuan, tapi harga bisa berubah
Landing page personal	SSG	Satu halaman, jarang di-update
Aplikasi chat realtime	SPA	State client berat
Portal berita	SSR / ISR	SEO penting, konten dinamis

Intinya: Ga ada satu metode yang paling benar. Tergantung seberapa sering konten berubah, seberapa penting SEO, dan seberapa kompleks interaksi user.

8. Tabel Perbandingan Framework

Semua angka perkiraan — yang penting trend-nya, bukan angka pastinya.

Aspek	React	Vue	Svelte
Dirilis	2013	2014	2019
Bundle size (Hello World)	~42 KB gzipped	~16 KB gzipped	~2 KB gzipped
Bundle size (aplikasi tipikal)	besar	sedang	kecil
Learning curve	Curam (JSX, hooks)	Landai (HTML-based)	Paling landai
Ecosystem	Terbesar	Besar	Sedang (tumbuh)
Job market	Paling banyak	Banyak (Asia)	Mulai ada
Hydration	Full hydration	Full hydration	No hydration
Reactivity model	Virtual DOM	Proxy (Vue 3)	Compile-time sig
TypeScript support	Bawaan	Opsional	Baik
Mobile	React Native	NativeScript, Weex	(via SvelteKit)

Penjelasan tiap kolom

Bundle size: Framework ringan ngirim lebih sedikit JS ke browser. Svelte dan Solid kompiler — hasil akhir kecil. Qwik malah kirim 0 JS awal (resumability). React banyak karena musti bawa Virtual DOM engine.

Learning curve: Vue dan Svelte pakai template mirip HTML — siswa yang udah tau HTML langsung bisa baca. React pake JSX (HTML di JS) — perlu adaptasi. hooks tambah kompleks.

Ecosystem: React punya library buat apa aja — UI component, testing, state management, animation. Vue juga kuat (terutama di Asia).

Svelte ekosistem lebih kecil tapi SvelteKit udah include router + SSR out of the box.

Hydration: Setelah SSR, React dan Vue harus "menghidupkan" HTML statis dengan ngejalanin JavaScript di client (hydrate). Svelte dan Solid gausah — mereka reactive dari awal. Qwik beda lagi: **resumable** — ga perlu ngulang eksekusi kode di client, cukup lanjutin dari state yang tersimpan.

9. Kesimpulan — Pilih yang Mana?

Ga ada framework terbaik. Yang ada adalah framework yang cocok buat situasi tertentu.

Panduan memilih:

Kalo...	Coba...
Mau kerja di perusahaan besar	React — paling banyak dicari
Mau yang sederhana, mirip HTML	Vue atau Svelte
Mau bundle paling ringan	Svelte atau Solid
Mau performa super tinggi	Solid (signal-based, ga ada Virtual DOM)
Mau yang paling baru & beda	Qwik (resumability = masa depan?)
Single developer / proyek sendiri	Pake apapun yang paling nyaman

Yang lebih penting: Pahami konsep component, reactivity, lifecycle, state management, routing. Begitu paham konsep ini, belajar framework baru tinggal belajar **sintaks** — butuh 1-2 minggu, bukan 6 bulan.

Semua framework pinjam ide satu sama lain. React pake hooks dari Svelte? Vue pake signal dari Solid? Svelte 5 pake runes mirip Solid? Ya — semuanya konvergen ke pola yang sama. **Belajarlah pola-nya, bukan framework-nya.**

Latihan (untuk dikerjakan sendiri)

1. Buat component counter (tombol + / -) di framework manapun — liat state, props, events di komponen itu
2. Identifikasi: mana state lokal? Mana yang perlu di-lift?
3. Coba pindahkan counter itu ke global store (Zustand / Pinia / Svelte store)

4. Tambah routing: halaman Home, About, Counter — pake router framework masing-masing
5. Ukur bundle size hasil build — bandingkan dengan framework lain

Referensi lanjutan:

- react.dev
- vuejs.org
- svelte.dev
- solidjs.com
- qwik.dev
- [Next.js vs Nuxt vs SvelteKit](#) — perbandingan meta-framework